

SIMATS ENGINEERING

Assessment Tool 2: Case Study — Answer Script

Course: Cloud Computing and Big Data Analytics (CSA15)

CO3: Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications (BL3)

Case Study 1: Smart Retail Inventory Management System

1. Understanding of the Case

The retail chain needs a centralized, always-available system that keeps stock data consistent across many physical stores. Key requirements drawn from the case are:

- Business requirement: real-time visibility of stock across all stores to avoid overstocking/understocking and to support demand forecasting.
- Technical requirement: POS systems and manager dashboards (clients) must talk to a centralized backend through Web APIs rather than direct database access.
- Data requirement: product catalogue, transaction logs, and analytics datasets must be durably stored and queryable — this points to cloud storage plus a database/data warehouse.
- Platform requirement: demand forecasting and analytics need a managed platform service (PaaS) instead of the retailer building ML infrastructure from scratch.
- Security requirement: store devices connect over public/semi-public networks, so VPNs, firewalls, and API gateways are required to protect data in transit and control access.

2. Application of Cloud Concepts

Mapping the case to the Service and Deployment models:

- **IaaS** — Virtual machines / auto-scaled compute instances host the core inventory service so capacity can grow during sales seasons.
- **PaaS** — A managed analytics/ML platform (e.g., AWS SageMaker / Azure ML / Google Vertex AI) is used for demand forecasting so the team focuses on models, not infrastructure.
- **SaaS-like dashboards** — Store manager dashboards are delivered as web applications accessed through a browser, needing no local installation.
- **Cloud storage** — Object storage (e.g., Amazon S3 / Azure Blob) holds product images and analytics datasets; a managed relational/NoSQL database stores transactional data and logs.
- **Web APIs** — RESTful APIs expose inventory operations (stock check, update, sale) so every POS terminal and dashboard uses the same, versioned interface.
- **API Gateway** — Centralizes authentication, rate limiting, and routing for all client requests, reducing duplicate security logic in each client.

3. Analysis (Trade-offs)

- **Centralization vs. latency**: a single central cloud region simplifies consistency but adds latency for distant stores; a Content Delivery Network (CDN) or regional read-replicas reduce this at the cost of extra complexity and sync overhead.
- **Consistency vs. availability (CAP theorem)**: stock counts should favour strong consistency to prevent overselling, while analytics data can tolerate eventual consistency, so a hybrid database strategy is justified.

- **Security vs. usability:** VPNs and mutual TLS increase protection but add setup overhead for thousands of store devices; API gateways with token-based auth (OAuth2/JWT) offer a lighter-weight alternative that scales better operationally.
- **Cost vs. elasticity:** auto-scaling compute controls cost during off-peak hours but requires careful configuration to avoid slow scale-up during flash sales.

4. Proposed Solution Design

A three-tier cloud architecture is recommended:

- **Client tier:** POS terminals and manager dashboards (web/mobile apps) communicate only through HTTPS REST APIs.
- **API/Platform tier:** An API Gateway routes requests to microservices (Inventory Service, Sales Service, Forecasting Service) running on auto-scaled VMs or containers (Kubernetes).
- **Data tier:** A managed relational database (e.g., Amazon RDS) stores live inventory and transactions; object storage holds bulk analytics datasets and logs; a managed ML/analytics platform (PaaS) consumes this data for demand forecasting.
- **Security layer:** Site-to-site VPN links stores to the cloud VPC; a Web Application Firewall and API gateway enforce authentication (OAuth2), rate limiting, and input validation; data is encrypted at rest and in transit (TLS 1.2+).
- **Monitoring:** Centralized logging and alerting (e.g., CloudWatch) tracks stock-sync failures and unusual API traffic.

This design is feasible because it uses standard managed services (reducing operational burden), scales horizontally for peak shopping periods, and isolates security concerns at the gateway/VPN boundary rather than in every client device.

Case Study 2: Cloud-Based Document Collaboration System

1. Understanding of the Case

- Business requirement: multiple users must edit the same document simultaneously (real-time collaboration), similar to Google Docs, with no data loss.
- Technical requirement: browser-based access (no installed client) implies a web-application front end using web sockets/APIs for live updates.
- Data requirement: documents need durable, distributed storage with version history so users can restore earlier states.
- Global requirement: users are geographically distributed, so low latency and high availability worldwide are essential.
- Security requirement: encryption, identity management, and fine-grained access permissions (owner, editor, viewer) are mandatory since documents may hold sensitive business data.

2. Application of Cloud Concepts

- **Web APIs / WebSockets:** REST APIs handle document CRUD operations (create, open, save, share); WebSocket or long-polling connections push real-time character-level edits between collaborators (operational transformation / CRDT-based sync).
- **Distributed cloud storage:** Object storage (e.g., Google Cloud Storage / Amazon S3) with cross-region replication stores document content and revision snapshots, giving durability and global reach.
- **Version control:** Each save creates an immutable revision object; a metadata database (NoSQL, e.g., DynamoDB/Firestore) tracks version chains so any past state can be restored.
- **PaaS:** A managed identity provider (e.g., Auth0, AWS Cognito, Azure AD) handles authentication/authorization instead of building a custom login system.
- **CDN:** Static front-end assets are cached at edge locations to minimize load time for users worldwide.
- **Deployment model:** Public cloud with multi-region deployment gives the elasticity and global footprint the case requires.

3. Analysis (Trade-offs)

- **Real-time sync vs. conflict resolution:** allowing simultaneous edits improves usability but requires conflict-resolution algorithms (Operational Transformation or CRDTs); this adds engineering complexity compared to simple lock-based editing.
- **Global availability vs. consistency:** multi-region replication improves latency and uptime but risks temporary inconsistency between regions; an eventual-consistency model with conflict resolution is an acceptable trade-off for a document editor.
- **Storing every revision vs. storage cost:** full version history improves recoverability but increases storage costs; a retention policy (e.g., keep frequent snapshots recently, coarser snapshots over time) balances the two.
- **Security vs. collaboration speed:** granular per-document permissions protect sensitive files but add an authorization check on every API call; caching permission decisions briefly can reduce overhead without materially weakening security.

4. Proposed Solution Design

- **Client tier:** Browser-based single-page application connects via HTTPS REST APIs for document management and a WebSocket channel for live edits.

- **Application tier:** Stateless microservices (Document Service, Collaboration/Sync Service, Sharing Service) run behind a load balancer across multiple cloud regions.
- **Data tier:** Document content stored in distributed object storage with cross-region replication; metadata and permissions stored in a globally distributed NoSQL database; a CRDT-based merge engine resolves simultaneous edits.
- **Security layer:** Managed identity service issues short-lived tokens (OAuth2/OpenID Connect); documents encrypted at rest (AES-256) and in transit (TLS); role-based access control (owner/editor/viewer) enforced at the API layer.
- **Performance layer:** CDN serves static assets; regional edge nodes minimize round-trip time for collaborative sessions.

This design is practical because it separates the concerns of storage, synchronization, and security into independently scalable services, and it uses proven managed platform components (identity, CDN, distributed database) rather than custom-building high-risk infrastructure such as global replication from scratch.

Case Study 3: Online Food Ordering Platform

1. Understanding of the Case

- Business requirement: a startup needs a scalable platform reachable from both web browsers and mobile apps, so the same backend must serve multiple client types.
- Technical requirement: frontend clients call backend services through RESTful Web APIs — a decoupled client-server architecture.
- Data requirement: menu images, invoices, and customer data must be stored securely, implying object storage for files and a secure database for personal/financial data.
- Scalability requirement: order volume can spike sharply (meal times, promotions), so the infrastructure must scale elastically — the case explicitly mentions VMs and managed Kubernetes.
- Security requirement: authentication tokens, HTTPS, and role-based access control (customer, restaurant, delivery agent, admin roles) are required to protect customer and payment-related data.

2. Application of Cloud Concepts

- **IaaS (VMs):** Used for services with predictable, steady load or for legacy components that are not yet containerized.
- **Managed Kubernetes (CaaS/PaaS):** Hosts containerized microservices (Order Service, Menu Service, Payment Service, Notification Service), enabling independent scaling and rolling deployments.
- **Web APIs:** RESTful endpoints expose menu browsing, cart/order management, and payment initiation to both the web app and mobile apps from a single backend.
- **Cloud storage:** Object storage holds menu images; a secure managed database (with encryption) stores invoices and customer profiles, separated from less-sensitive catalogue data.
- **Security services:** An API gateway validates JWT/OAuth2 tokens on every request; HTTPS/TLS is enforced end-to-end; role-based access control differentiates customer, restaurant-owner, and delivery-partner permissions.
- **Auto-scaling:** Kubernetes Horizontal Pod Autoscaler adjusts the number of running service instances based on real-time order traffic.

3. Analysis (Trade-offs)

- **Microservices on Kubernetes vs. simplicity:** containerized microservices give fine-grained scalability (e.g., scale only the Order Service during a lunch rush) but increase operational complexity versus a single monolithic VM-based app; for a startup, managed Kubernetes (EKS/AKS/GKE) offsets this by handling cluster operations.
- **Cost vs. peak readiness:** keeping capacity provisioned for peak meal-time load wastes money at off-peak hours; auto-scaling and serverless functions for spiky workloads (e.g., notifications) reduce idle cost while still meeting demand.
- **Token-based auth vs. session-based auth:** stateless JWT tokens scale better across many API instances than server-side sessions, at the cost of needing careful token expiry/revocation handling.
- **Storing images in object storage vs. database:** object storage is cheaper and faster for large binary files (images) than storing them as database blobs, but requires an extra layer to keep image references consistent with menu records.

4. Proposed Solution Design

- **Client tier:** Web app and mobile apps consume the same versioned REST API set, ensuring consistent behaviour across platforms.

- **Application tier:** Managed Kubernetes cluster runs containerized microservices (Menu, Order, Payment, Notification) behind an API gateway; VMs handle any legacy or steady-load components.
- **Data tier:** Object storage for menu images; an encrypted managed database (e.g., PostgreSQL/MySQL on a managed service) for customer data and invoices; a caching layer (e.g., Redis) speeds up menu browsing.
- **Security layer:** All traffic over HTTPS; OAuth2/JWT-based authentication; role-based access control at the API gateway; PCI-DSS-aligned handling for payment data (tokenized card details, not stored directly).
- **Scalability layer:** Horizontal Pod Autoscaling in Kubernetes plus cloud load balancers absorb meal-time traffic spikes; a message queue (e.g., SQS/RabbitMQ) decouples order placement from downstream processing (kitchen notification, delivery dispatch).

This design is feasible and justified because it matches each requirement from the case to a specific, well-established cloud pattern: containers/Kubernetes for elastic microservices, object storage for images, a secure managed database for sensitive records, and token-based security enforced uniformly through an API gateway across both web and mobile clients.

End of Answer Script